Chapter 6. Interactivity: Gestures, Forms, Navigation

This chapter is the study material of the sixth lecture. Chapter 4 clarified *what* a widget is, Chapter 5 — how it gets its size and its place on the screen. The practical consequence: your L2 application looks, at this moment, exactly like the final product. And it does *nothing*. This is the chapter in which the application starts to respond, in three steps: first the **touches** — callbacks as function values and the gestures with or without visual feedback (6.2), then the **immediate response** — `SnackBar` and the dialogs with a result (6.3); then come the two centerpieces: **forms with validation** — `Form`, `TextFormField`, the `validator` idiom (6.4) — and **address-based navigation** — from the `Navigator.push` stack to the `go_router` route map, with `go` vs `push` and path parameters (6.5); at the end we tie everything into the complete flow that lab work L3 requires (6.6). The whole chapter is the direct theoretical support of the L3 guide — which begins right now and adds, step by step, the persistent tabs (`ShellRoute`), *not* taught here. What is **not** in this chapter: state *shared* between screens — the cart that remembers, the list that actually grows after submit — is Chapter 7 material; real authentication, with a backend, is L5 material. At the end: the summary and the self-assessment exercises.

6.1 The L2 application: looks perfect — and does nothing

Reread the letter of the L2 guide: your application is a “visually functional mock-up” — “it looks exactly like the final product, but the data is hardcoded and the buttons may do nothing”. More than that, on the authentication screen validation was explicitly *forbidden*: “no `Form`, no `validator:`, no checking of the fields whatsoever, no logic on the button press”. That ban expires in this chapter.

Open your application and try, with a user’s cold eye, three things. Press “Sign in” — nothing. Tap a card in the list — nothing. Type “abc” into the password field — it swallows it without a word. It is not a defect: it is exactly the L2 contract, a mock-up that *looks*. But the distance between the mock-up and the application is the distance between this chapter and the ones before it — and it is measured in three questions:

1. **How do I tie a piece of code to a touch?** — callbacks and gestures (6.2), plus the visible confirmation that the touch was received (6.3).
2. **How do I refuse bad data and receive the good data?** — forms with validation (6.4).
3. **How do I get from one screen to another — and back?** — navigation, imperative first, then by addresses (6.5).

Translate them into the language of your theme from the catalog, because that is precisely what your theme’s L3 bullet asks of you: at MediShop — catalog → detail → cart → placing the order with a validated phone; at FitTrack — journal → the session form with the numeric duration; at WalletMate — the positive amount and the mandatory category. Each of these flows is assembled, piece by piece, from the material of this chapter.

One convention before we start, as in Chapters 4 and 5: the fragments assume `import 'package:flutter/material.dart';` at the top of the file and can be tried in DartPad, which runs Flutter too. The navigation fragments of 6.5 additionally require `import 'package:go_router/go_router.dart';` (we install the package right there), and the typing filters of 6.4 — `import 'package:flutter/services.dart';`.

6.2 Callbacks and gestures: how a widget listens

onPressed: the function in the button's pocket

You have been writing `onPressed` since Chapter 4, in passing. Today we look it in the face, with the underlying question: what *type* is the thing we put in `onPressed`? The answer: a **function**. In the API the type is called `VoidCallback`, and it is just an alias: `void Function()` — a function with no parameters that returns nothing. Here Chapter 2 pays off directly — the "functions are values" section: in Dart a function is passed as an argument like any other value. The button does not "know" what it does — it keeps a function in its pocket and calls it when it is touched:

```
// C2: "functions are values" — the callback is one of them
FilledButton(
  onPressed: () {
    debugPrint('order placed');
  },
  child: const Text('Place the order'),
)
```

The anonymous form `() { … }` suits short logic, written in place. When the function already exists as a method of the screen, pass its *reference*:

```
onPressed: _submitOrder,          // ✓ you pass the function itself
onPressed: () => _submitOrder(),  // ✓ a lambda that calls it on touch
```

The bridges from your world: in Swing you wrote `button.addActionListener(e -> …)` — an anonymous class or a lambda; in C# — `button.Click += OnSubmit;`. The contract is the same: "when the event happens, call my code". The difference of dialect: in Flutter the callback is a *constructor parameter*, handed over together with the widget's description, not a subscription made later on a live object — one more consequence of the widget being immutable configuration (Chapter 4).

The parentheses trap: _submitOrder() runs at build

The difference between the two lines below is one pair of parentheses — and a chasm of meaning:

```
onPressed: _submitOrder,    // ✓ a reference: called on TOUCH
onPressed: _submitOrder(),  // ✗ a call: runs NOW, at build
```

The parenthesized variant does not “wire up” the function — it *calls* it on the spot, during `build`, and tries to hand the button its *result*. And `build`, you know from Chapter 4, runs often: the order would get “placed” on every repaint, not on the press. In the typical case Dart catches you: `_submitOrder()` has type `void`, and `void` does not fit where a function is required — a compile error. But understand *why* it is wrong, not just that “it doesn’t compile”, because there is a case that compiles and hurts: a function that *returns a function*.

```
VoidCallback makeSubmitHandler() {
  debugPrint('this line runs at BUILD — on every repaint');
  return () => debugPrint('only this one runs on touch');
}

// compiles perfectly: the call returns a VoidCallback — the type fits
FilledButton(
  onPressed: makeSubmitHandler(),
  child: const Text('Submit'),
)
```

No error message anywhere — just the body of `makeSubmitHandler` executed on every `build`, with whatever side effects it has. The hygiene rule: to `onPressed` (and to any callback parameter) you pass a *function*, not the result of a call; if you need parentheses, wrap the call in a lambda: `() => _submitOrder()`.

null instead of a function: the button disables itself

Look at the type once more: the parameter is in fact `VoidCallback?` — *nullable*, with everything you learned about `?` in Chapter 2. And `null` has a design meaning here — it is not an accidental missing value: **`onPressed: null` means “the button has nothing to do” — and Flutter draws it gray and mutes it by itself**, without a single line of styling code on your part. Hence the conditional idiom:

```
FilledButton(
  // a function when allowed, null when not: the button disables itself
  onPressed: canSubmit ? _submitOrder : null,
  child: const Text('Place the order'),
)
```

“The button works only when you’re allowed to” — in three words of code, with no manual bookkeeping of colors and states. You will reuse it at L3 on the submit buttons. And a one-line generalization, worth keeping in mind for the multiple-choice tests: not all callbacks are parameterless — the `onChanged` of a text field, for instance, receives the current text and has type `ValueChanged<String>`, that is, `void Function(String)`. The mechanism is identical; only the signature differs.

The L2 card wakes up: ListTile already has onTap

Buttons have `onPressed` — but your list card? The good news, with a direct pointer to L2: the reusable card is a `ListTile`, and `ListTile` already *has* `onTap` — its interactivity is literally a parameter. The L2 guide even suggested, at Task 5, parameterizing the card "through the constructor (the displayed object, possibly an `onTap`)" — the slot deliberately left empty then gets filled now:

```
class RecordCard extends StatelessWidget {
  final VinylRecord record;
  final VoidCallback? onTap; // the card does not decide WHAT happens on tap

  const RecordCard({super.key, required this.record, this.onTap});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        onTap: onTap, // ListTile already listens — we just hand it the function
        title: Text(record.title),
        subtitle: Text(record.artist),
      ),
    );
  }
}
```

Notice the design decision, worth more than the code: the card receives the callback *from outside* and does not know what happens on the touch — today a `SnackBar`, in 6.5 a navigation to the detail, and the card itself is not touched at all in between. It is the same separation every well-raised reusable component has.

GestureDetector: any gesture, on any widget — but mute

For the widgets that do *not* have `onTap` — an image, a `Container`, an entire row — there are two wrappers. The first, the almighty one: `GestureDetector` catches practically any gesture on any widget:

```
GestureDetector(
  onTap: () => debugPrint('a short tap'),
  onLongPress: () => debugPrint('a long press — where the context menu lives'),
  child: const CircleAvatar(child: Text('K')),
)
```

Beyond `onTap` and `onLongPress` there are `onDoubleTap` and a whole family of drag gestures (`onHorizontalDragEnd` and its relatives), out of which swipes are built — we only mention them; when you need "delete the item with a swipe", look up the dedicated widget `Dismissible`, which packages the gesture together with its animation. The price of the power: `GestureDetector` is completely **visually mute**. No ripple, no pressed state — the user gets no sign that the touch was

seen, and on a phone that absence reads immediately as “broken app”. The bridge: it is Flutter’s version of “any component can receive a `MouseListener`” from Swing — just as universal, just as mute.

InkWell: tap plus the Material ink — and the opaque-child trap

The second wrapper, the civilized one: `InkWell` catches the tap *and* draws the Material ink effect — the ripple that the buttons and the `ListTile` have too. One condition and one trap, both tied to the same mechanism. The condition: `InkWell` requires a `Material` ancestor, because the ink is painted on that surface; inside a `Card` or on any screen with a `Scaffold` you get it for free — which is why in your project it will work right away.

The trap follows from the same sentence, read carefully: the ripple is painted **on the Material underneath** — that is, *under* the widget’s child. An opaque child that fills the whole area — the typical case: the card’s cover image — covers the ink completely:

```
// ✗ the tap WORKS — but the ripple is never seen:
//   the opaque image covers the Material it is painted on
InkWell(
  onTap: _openDetail,
  child: Image.asset('assets/images/cover.jpg',
      height: 160, fit: BoxFit.cover),
)
```

Nothing crashes, nothing warns — only the feedback disappears, exactly the thing you chose `InkWell` for. For images there is the dedicated tool: `Ink.image` paints the image *onto* the Material, not on top of it, so the ink stays visible over the image:

```
// ✓ the image becomes part of the Material: the ripple shows over it
Card(
  clipBehavior: Clip.antiAlias, // clips the image's corners to the card
  child: InkWell(
    onTap: _openDetail,
    child: Ink.image(
      image: const AssetImage('assets/images/cover.jpg'),
      height: 160,
      fit: BoxFit.cover,
    ),
  ),
)
```

The same trick exists for decorated backgrounds: `Ink` with a `decoration`, in place of an opaque `Container`. Do not memorize the pairs — memorize the mechanism: *the ink lives on the Material; whatever is opaque above the Material hides it.*

The three-step rule

With all the pieces on the table, the hygiene rule of touch — in the order you *try*:

1. **the dedicated widget** — a button, `ListTile`, `IconButton`: they have the callback, the ripple, and the accessibility semantics (screen readers announce them as buttons);
2. **`InkWell`** — when you wrap something tappable that has no callback of its own: you keep the visual feedback;
3. **`GestureDetector`** — last: when you need rich gestures (long-press, double-tap, drag) and you explicitly own the feedback.

Whoever jumps straight to `GestureDetector` "because it works anywhere" loses the ripple — that is, loses the user.

6.3 Immediate feedback: SnackBar and the dialogs

The touch has been caught — the user must *learn* that something happened. The tools are chosen by a single question: **does it ask for a decision, or does it merely announce?**

SnackBar: the passing announcement

`SnackBar` is the announcement that blocks nothing: it appears at the bottom, stays a few seconds, disappears by itself. "Product added to the cart", "assignment saved" — confirmations, not conversations:

```
ScaffoldMessenger.of(context).showSnackBar(
  const SnackBar(content: Text('Added to cart')),
);
```

Notice *whom* it is requested from: not the screen's `Scaffold`, but a **`ScaffoldMessenger`**. The `.of(context)` lookup you know from Chapter 4 (Section 4.8): from your position in the tree, upward, to the object installed above — the same mechanism as `Theme.of`. And the messenger's position explains *why* it exists: `MaterialApp` installs it **above all the screens**. If the announcement belonged to the `Scaffold`, it would die together with the screen: you show "Saved" and immediately close the form — the message would vanish halfway through. The messenger, sitting above, keeps the announcement alive even across a navigation: "Saved" shows serenely on the screen you came back to. (A `SnackBar` can also carry a short action — `action: SnackBarAction(label: 'Undo', …)` — a mention for later; today the announcement is enough.)

showDialog: the blocking question — a Future, as in Chapter 3

When you genuinely need a decision — "delete the record?" — an announcement is not enough: you must *stop* the flow until the user answers. The tool is `showDialog`, and its signature is a small recap of Chapter 3: it returns a **`Future<T?>`** — the dialog is an asynchronous operation, and the `await` receives its value only when the user presses something:

```dart
Future<void> _confirmDelete(BuildContext context) async {
  final ok = await showDialog<bool>(
    context: context,
    builder: (dialogCtx) => AlertDialog(
      title: const Text('Delete the record?'),
      content: const Text('This action cannot be undone.'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogCtx, false),
          child: const Text('No'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(dialogCtx, true),
          child: const Text('Yes, delete'),
        ),
      ],
    ),
  );
  if (ok == true && context.mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Deleted from the collection')),
    );
  }
}
```

The fragment is short, but every line carries a rule — we take them one by one, because all four get checked at the L3 defense.

Closing is a `pop` with a result. The dialog is, technically, a "page" pushed over the screen — which is why it closes with `Navigator.pop`, and the second argument is the *result* that reaches the `await`: `true` from "Yes", `false` from "No".

`ok` can be `null` — always handle it. The returned type is `Future<bool?>`, not `Future<bool>`, and the question mark is not pedantry: the user can dismiss the dialog *sideways* — a tap outside it, the back gesture — and then no button was pressed: you receive `null`. That is why the correct condition is `ok == true`, not `ok!` — Chapter 2's null safety protects you here too, if you let it. `if (ok!)` would crash in exactly the scenario you did not test.

The dialog closes with `dialogCtx`, not with `context`. Look at the `builder`: it receives its own context — we named it `dialogCtx` — and *that* is what we `pop` with. Here the promise from 4.8 gets paid: "whose context is it" becomes a serious question. `Navigator.pop(ctx)` climbs from `ctx` to the first `Navigator` and removes its topmost page. From `dialogCtx`, the path is guaranteed to lead to the navigator the dialog was opened on. From the screen's `context`, today, it would happen to be the same one — but at L3, when the persistent tabs put your screens under *nested navigators* (`ShellRoute`), the screen's context leads to another navigator, and the `pop` would close *something else* — the screen under the dialog, say, with the dialog left orphaned on top. The correct habit costs zero today and saves you an hour of debugging then: **the dialog closes with the builder's context.** (And the taxonomic reminder from 4.8: `Navigator.of` only *resembles*

`Theme.of` — it climbs the tree looking for a `Navigator`'s State, without subscribing; it does not stand on `InheritedWidget` and does not rebuild you when something changes.)

After an `await`, the context is reused only under `context.mounted`. While the dialog stayed open, the screen beneath it could have died (a programmatic navigation, a restart). The analyzer knows the trap and has a dedicated lint — `use_build_context_synchronously` — which will scold you the first time you reuse a `BuildContext` after an `await` without checking. The cure is right in the fragment: `context.mounted` — "is my element still in the tree?". At L3 you hit the lint exactly once and the reflex stays.

The bridges: `JOptionPane.showConfirmDialog` from Swing, `MessageBox.Show` from WinForms — the same dialogs with a result, with one fundamental difference: there, the call *blocked the thread*; here everything stays asynchronous — the UI is alive, the animations run, only *your flow* waits at the `await`. It is exactly the "mobile is not allowed to wait" discipline from 3.6, applied to dialogs.

The section's pocket rule: **just announcing — `SnackBar`; asking for a decision — a dialog with `await`. And the "dismissed outside" `null` always gets handled.**

6.4 Forms with validation — centerpiece 1

TextField and TextFormField: the raw field and its integrated sibling

Your static login from L2 has two `TextFields`. `TextField` is the *raw* field: it draws, receives keystrokes, shows its `decoration` — that's all. Its sibling, **`TextFormField`**, is `TextField` plus the form integration: it accepts `validator` and `onSaved` and knows how to work in a team under a `Form`. The practical rule is simple: a lone field (a search, a filter) — `TextField`; a field in a form with rules — `TextFormField`. At L3, all the fields of your forms become `TextFormField`.

TextEditingController: who owns the text — and who releases it

The first practical question: how do you *read* what the user typed? The idiom-answer: through a **`TextEditingController`** — the object that owns the field's text. You read it (`_emailCtrl.text`), set it, clear it after submit. And here comes the architectural consequence, tied directly to Chapter 4: the controller is an object with a life of its own — **it is not created in `build`**. `build` runs often (4.1); a controller created there would be replaced on every repaint, typed text and all. The controller therefore lives in the `State` — which means the login screen becomes a `StatefulWidget`:

```dart
class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  // they live WITH the screen — fields of the State, not variables in build (C4)
  final _emailCtrl = TextEditingController();
  final _passwordCtrl = TextEditingController();

  @override
  void dispose() {
    _emailCtrl.dispose(); // who creates, releases
    _passwordCtrl.dispose();
    super.dispose();
  }

  // the build comes in the next sections
}
```

Reread the lifecycle from 4.4 with your eyes on this fragment: `createState` → the State's fields are initialized → `build` many times → `dispose` at the screen's death. And in `dispose` you **release the controller** — one `dispose()` per controller, before `super.dispose()`. Without it, every entry into and exit from the screen leaves behind a live controller, listeners and all — a silent memory leak, which the L3 review hunts explicitly. The bridge for the .NET folks is literal: `IDisposable` — and for an object's fields, you are the one who calls `Dispose` from the owner's `Dispose`. The same contract: *who creates, releases.*

And the warning that keeps Chapter 7 at bay, said firmly: what we did here is **not "state management"**. It is the *local* state of a text field — it lives and dies with the screen, nobody else sees it. The question "what do I do when several screens want the same state?" stays open until Chapter 7 — do not install any state library for a form: the form manages on its own.

Form and GlobalKey: the Chapter 4 promise, paid off

A real form has several fields and a single command: "validate yourselves, all of you". The wrapper that makes the team is called **`Form`** — an invisible widget that takes under its command all the `TextFormFields` beneath it: it can validate them all at once, save them, reset them.

To give it the command, you need a handle to its *state* — and here the promise from 4.7 gets paid. Ordinary keys (`ValueKey`, `ObjectKey`) help the diff algorithm identify widgets across rebuilds. **`GlobalKey`** is the relative that additionally does one thing no ordinary key does: it gives you *access* to the widget's state, from anywhere — `_formKey.currentState`. Why is a *global* one needed precisely here? Look at the tree:

```
LoginScreen                ← the screen's context is HERE, up top
  └─ Form (key: _formKey)  ◄── the key: the only bridge DOWNWARD
      └─ Column
          ├─ TextFormField (e-mail)
          ├─ TextFormField (password)
          └─ FilledButton – onPressed: a closure with the SCREEN's context
```

The validation command leaves from the button's `onPressed`. Its closure captured the screen's `context` — which sits *above* the `Form`. But all the `.of(context)` lookups from 4.8 climb *upward*: `Form.of`, called with this context, has no way of finding a `Form` sitting further *down*. (The ceremony alternative would be a `Builder` that manufactures a context for you under the `Form` — it can be done, but nobody prefers it here.) The global key is the bridge in the direction the context does not lead: we keep the handle in the State and reach the `FormState` directly, from anywhere. That much — and just as important, what `GlobalKey` is *not*: it is not the everyday tool; it has its bookkeeping cost in the framework and a single canonical place of use in the whole semester — the form.

validator: it returns the error text — and null is the good news

The heart of the form and the most counterintuitive idiom of the chapter. Read the signature out loud: `validator` receives `String?` — the field's current text — and **returns `String?`**: the *error text* if something is wrong, or **`null` if all is well**.

```dart
final _formKey = GlobalKey<FormState>(); // in the State, next to the controllers
```

```
Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      controller: _emailCtrl,
      keyboardType: TextInputType.emailAddress,
      textInputAction: TextInputAction.next,
      decoration: const InputDecoration(labelText: 'E-mail'),
      validator: (value) {
        if (value == null || value.trim().isEmpty) {
          return 'Enter your e-mail';
        }
        if (!value.contains('@')) {
          return 'The e-mail does not look valid';
        }
        return null; // null = VALID — no error to report
      },
    ),
    TextFormField(
      controller: _passwordCtrl,
      obscureText: true, // you have it from L2 — now you know its family
      textInputAction: TextInputAction.done,
      decoration: const InputDecoration(labelText: 'Password'),
      validator: (value) => (value == null || value.length < 6)
          ? 'The password must be at least 6 characters'
          : null,
    ),
  ]),
)
```

Yes, it is backwards from instinct: "the function returned `null`" sounds like a problem, but here `null` means "*no error to report*". Think of the validator as a function that returns the list of problems found: the empty list is the good news. Dictate the idiom to yourself until it becomes a reflex — **`null` = valid; any `String` = the text that will appear, in red, under the field** — because it is a sure multiple-choice topic on Test 1, in exactly this phrasing.

And notice what you did *not* write: not one line of display code. `TextFormField` draws its own error under the field, in the `error` color of the Chapter 5 scheme — the text you return lands in the field's `InputDecoration`, which is why it settles in nicely, in the theme's style.

The bridges: Bean Validation from Java (`@NotNull`, `@Size`) and DataAnnotations from .NET (`[Required]`, `[StringLength]`) — the same idea of a rule attached to the field. The difference of temperament: there, the rules are declarative annotations from a given set; here, the rule is a *function*, written in place, with the language's full power inside it — a regex, a comparison against another controller, anything you can express in Dart. In the language of the themes: MediShop's "mandatory phone", FitTrack's "numeric duration", WalletMate's "positive amount" — each is a validator of 3–5 lines. A numeric one, as a model — with `int.tryParse`, the case study from 2.4:

```
TextFormField(
  controller: _durationCtrl,
  keyboardType: TextInputType.number,
  decoration: const InputDecoration(labelText: 'Duration, minutes'),
  validator: (value) {
    final minutes = int.tryParse(value ?? ''); // null if not a number (C2)
    if (minutes == null) return 'Enter a whole number';
    if (minutes <= 0) return 'The duration must be positive';
    return null;
  },
)
```

The submit: validate() → controllers → action

The circuit closes in the button's `onPressed`:

```
FilledButton(
  onPressed: () {
    // runs ALL the validators in the Form at once → bool
    final valid = _formKey.currentState!.validate();
    if (!valid) return; // the errors are already on screen — do nothing more

    // the data: straight from the controllers
    final email = _emailCtrl.text.trim();
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Hello, $email')),
    );
    // at L3, navigation follows here — Section 6.6
  },
  child: const Text('Sign in'),
)
```

`validate()` is the team command: it runs *all* the validators in the `Form` and returns `true` only if every answer was `null`. The visible effect deserves underlining: all the fields with problems bloom red *simultaneously* — the user sees everything there is to correct at a glance, instead of discovering the errors one at a time, submit after submit.

The `!` after `currentState` is explained by Chapter 2 in full: `currentState` is nullable — the key might not (yet) be attached to a mounted `Form`. Here, though, we are in the `onPressed` of a button that exists only *under* that `Form`: the guarantee is structural, which is why we allow ourselves the `!`. It is the kind of judgment null safety explicitly asks of you: not "I put a `!` to silence the analyzer", but "I know why non-null is guaranteed here".

There is also an alternative path, without controllers: each `TextFormField` can have `onSaved`, and `_formKey.currentState!.save()` calls them all — the fields *pour* their values into your variables:

```
String _title = '';

TextFormField(
  decoration: const InputDecoration(labelText: 'Title'),
  validator: (value) => (value == null || value.trim().isEmpty)
      ? 'The title is required'
      : null,
  onSaved: (value) => _title = value!.trim(),
),
```

```
// at submit: validate first, then save
if (_formKey.currentState!.validate()) {
  _formKey.currentState!.save(); // calls all the onSaved callbacks
  // _title now holds the field's value — without any controller
}
```

The practical choice: at 2–3 fields, the controllers are more direct (and you need them anyway if you read or set the text outside the submit); on bushy forms, `onSaved` spares you a whole list of fields in the State and the matching `dispose`s. The L3 grading scheme does not impose a mechanism — it requires that validation *exist* and be *visible*.

The field's toolkit: keyboards, filters, and the rest of the parameters

The small toolkit that separates a "student" form from one that feels professional. It is not for memorizing — it is for *consulting*, field by field:

Parameter	What it does	Typical example
`keyboardType`	requests the keyboard suited to the field	`TextInputType.emailAddress` / `.phone` / `.number`
`obscureText: true`	hides what is being typed	the password — exactly your field from L2
`inputFormatters`	filters *what can enter* the field	`FilteringTextInputFormatter.digitsOnly` — the phone
`textInputAction`	the keyboard's action button	`.next` between fields, `.done` on the last one
`decoration`	label, hint, icon — and the error's place	`InputDecoration(labelText: 'Phone')`
`maxLength` / `maxLines`	the field's limit and height	the review, the description — the long fields

Two comments beyond the table. First: on a phone, *the keyboard is yours* — `keyboardType` requests the variant suited to the field (`emailAddress` puts `@` within reach, `phone` and `number` open the digits), and the user feels the difference from the first touch; at the L3 defense it is the first thing checked. Second, a difference of philosophy that deserves a rule of its own:

`inputFormatters` prevents *during* typing; `validator` judges *after*. The formatter simply does not let a letter into the phone field:

```dart
// requires import 'package:flutter/services.dart';
TextFormField(
  controller: _phoneCtrl,
  keyboardType: TextInputType.phone,
  inputFormatters: [FilteringTextInputFormatter.digitsOnly],
  decoration: const InputDecoration(labelText: 'Phone'),
)
```

— but it cannot judge *domain rules*: “the positive amount”, “the date in the future”, “at least 6 characters” can only be seen once the text exists. A good form uses both: the formatter keeps the field clean, the validator judges the rule. The habit to acquire for L3: every field of the form deserves 30 seconds of thought — which keyboard, which filter, which validator.

6.5 Navigation — centerpiece 2: from the stack to addresses

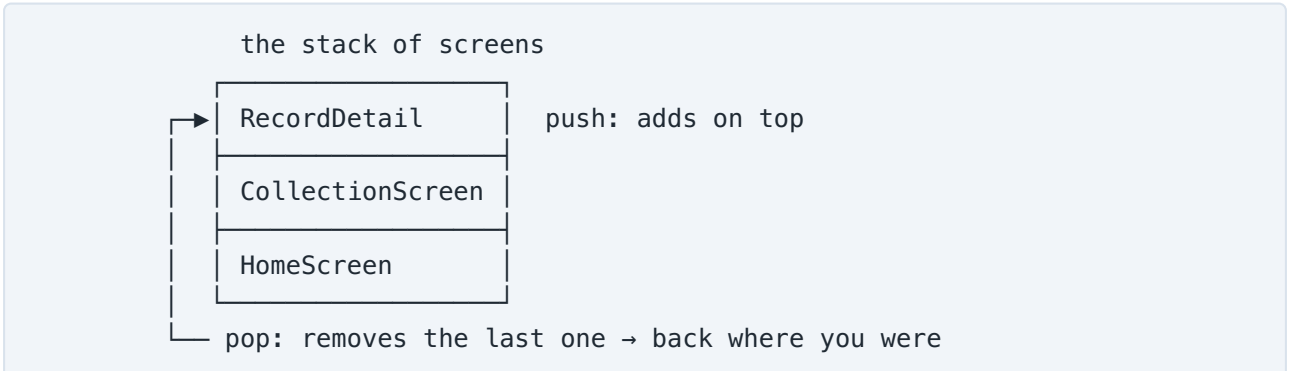
The stack of screens: push adds, pop removes

We start from what you *have*: Task 6 of L2 gave you `Navigator.push` with `MaterialPageRoute` — the “temporary scaffolding”, as the guide called it, promising that at L3 we replace it. We keep our word — but first let us say firmly what is *healthy* in the scaffolding, because the model stays:

```dart
// L2, Task 6 — the scaffolding: the imperative navigation command
Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const RecordDetailScreen(id: 7)),
);

Navigator.pop(context); // back — programmatically
```

The screens live on a **stack**: `push` puts one on top, `pop` removes the topmost. The back arrow in the `AppBar` and the phone’s back gesture do exactly that — a `pop`:



The bridge to the old Android course is direct: `startActivity(new Intent(this, DetailActivity.class))` — the same imperative philosophy, the same back stack; whoever

caught XAML recognizes `NavigationService.Navigate`. The stack model is correct and it stays — including under the modern tools.

The limit of the imperative: the current place has no name

Now the question that motivates the rest of the section: **where *am* I in the application?** The imperative stack's answer: "no idea — some pushes happened". The current place has no *name* — it is merely the result of a history of commands scattered through the screens' files. The consequences hurt exactly where the industry has moved:

- **the direct link**: "open product 7 from the notification" — has nowhere to enter: no screen has an address, so there is no door;
- **the web**: Flutter compiles for the browser too, and there the URL bar must *reflect* the place — impossible when the place has no name;
- **state restoration**: the system kills the application in the background; on return, "where was the user?" needs an answer that can be saved — a stack of commands executed at some point is not an answer.

The solution is a paradigm shift we have been repeating since Chapter 4: instead of **commanding the journey**, you **describe the map**. You declare the routes with addresses, and the framework builds the stack. Sound familiar? It is the same imperative → declarative move as with the UI — there you described *what the screen looks like*, not the drawing commands; here you describe the application's *topology*, not the navigation commands. The ecosystem's established tool for this style is called `go_router`.

go_router: a package — and the pub.dev ritual

The first observation, before any code: `go_router` does **not ship with the SDK**. It is a package of the ecosystem, published on **pub.dev** — the package registry of the Dart world — and maintained within the Flutter project. You install it like any package:

```
flutter pub add go_router
```

The command writes the dependency into `pubspec.yaml` (the file you know from assets, Chapter 5) and downloads it; the exact version is pinned by `pubspec.lock`, so the whole team compiles with the same libraries — nothing to memorize, no version number to remember. It is **the course's first external package**, so the ritual deserves a second of solemnity, because it repeats exactly: at Chapter 8 for `http`, at 9 for `drift`, at 12 for `firebase`. *pub.dev is the registry, `pub add` is the gesture, `pubspec.yaml` is the ledger.* The bridge: Maven Central + `pom.xml` in Java, NuGet + `.csproj` in .NET — the same model, another registry.

The route map — and the wiring that replaces home:

The map is a `GoRouter` object, usually in the project's `lib/router.dart`, with the list of routes: each `GoRoute` ties a **path** — an address — to a `builder` that returns the screen:

```dart
// lib/router.dart — the application's map: every screen gains an ADDRESS
final router = GoRouter(
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const CollectionScreen(),
    ),
    GoRoute(
      path: '/records/:id', // variable segment — next section
      builder: (context, state) {
        final id = int.parse(state.pathParameters['id']!);
        return RecordDetailScreen(id: id);
      },
    ),
    GoRoute(
      path: '/login',
      builder: (context, state) => const LoginScreen(),
    ),
  ],
);
```

The wiring happens in `main.dart`, and it brings a change many people miss: `MaterialApp` becomes **`MaterialApp.router`**, and the `home:` parameter from L2 **disappears** — whoever keeps both gets an error. The start screen is from now on the `'/'` route (or, if you want another, the `GoRouter`'s `initialLocation` parameter — the L3 guide starts the application in `/login` exactly this way):

```dart
class VinylShelfApp extends StatelessWidget {
  const VinylShelfApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      title: 'Vinyl Shelf',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF5D4037)),
      ),
      routerConfig: router, // home: is gone — the map decides the start
    );
  }
}
```

Two things to say so it does not look like magic. First: **under the hood it is still `Navigator`** and still the stack from the beginning of the section — `go_router` does not replace the model, it changes its *source*: the stack is built from your declaration, not from commands scattered through seven screen files. Second, practical: `router` is a global object, built exactly once at startup — hot reload (r) redoes the widgets' builds, but does not re-run already-evaluated global initializers (the Chapter 4 rule); any change to the route map therefore requires a restart (R).

The immediate gain, even before deep links and the web: **the application's whole topology reads from a single file.** At the L3 defense you are asked exactly that — to open `router.dart` and explain every route; compare that to hunting `push`es through all the screens.

context.go vs context.push: replace or add

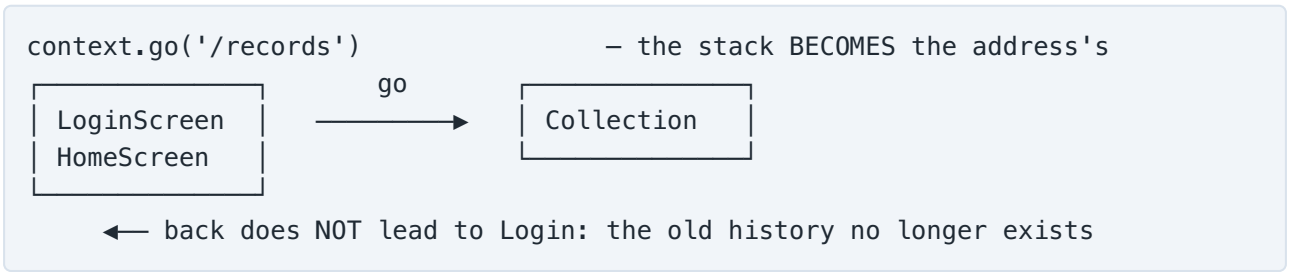
Once the map is wired up, navigation from code goes through the extensions `go_router` puts on `context` — and here is the pair of methods you will mix up for a week, then never again, if you grasp it *on the stack*, not from definitions. Both take an address; what differs is what happens to the stack.

`context.push('/records/7')` — adds. The address's screen goes *on top*; underneath, everything that was there remains. The back arrow appears by itself in the `AppBar`, and `pop` returns you exactly where you were:



It is the gesture for "I go in and come back": list → detail, opening a form.

`context.go('/records')` — you go to the address. The stack above is discarded and **becomes the one the address describes in the route map**. There is no "back" to what was before, because that history no longer exists:



The phrasing deserves precision, because the wrong version circulates: `go` does not simply "empty the stack" — it *rebuilds* it after the address. For top-level routes, as in the drawing, the result is a single screen; for the *nested* routes of the map, the stack becomes the chain of the address's parents — you will see the construction at L3, where the detail becomes a child route of the list. The pocket rule below, however, covers everything you do this semester.

The textbook case, checked explicitly at the L3 defense: **after authentication — `go`.** The user signs in and leaves for the main screen; the login must *vanish* from the stack. With `push`, the back button would bring the already-signed-in user back… into the login screen — the classic mistake, in every cohort. And the programmatic return — the "Cancel" button of a form opened with `push` — is `context.pop()`; the arrow in the `AppBar` calls the same function.

The final rule, to dictate: **you want to be able to come back — `push`; you want NO way back — `go`.**

Path parameters: /records/:id — the id travels, the screen does the lookup

The last piece: the detail must know *whom* it displays. The idiom of address-based routing: **the address carries the data.** In the map, the `:id` segment is variable — `/records/:id` catches `/records/7`, `/records/42`, anything; in the `builder`, `state.pathParameters['id']` hands you the value:

```
GoRoute(
  path: '/records/:id', // a variable segment in the path
  builder: (context, state) {
    final id = int.parse(state.pathParameters['id']!); // String! → int
    return RecordDetailScreen(id: id);
  },
),
```

Two subtleties that always get lost. First: the value is **always a `String`** — an address is text; if the model wants an `int`, the conversion is yours (`int.parse` — Chapter 2), it does not happen by itself. Second: the `!` after `pathParameters['id']` is structurally justified, as with the forms — on *this* route, the map guarantees that the `:id` segment exists; on another route, the same expression would be a lie.

Who sends it? The card from 6.2, with the id interpolated into the address — Chapter 2's string interpolation, at work:

```
RecordCard(
  record: record,
  onTap: () => context.push('/records/${record.id}'), // I go in and come back
),
```

And the detail screen **looks up its own object**:

```
class RecordDetailScreen extends StatelessWidget {
  final int id;
  const RecordDetailScreen({super.key, required this.id});

  @override
  Widget build(BuildContext context) {
    // today: a lookup in the mock list (firstWhere — C2);
    // at L4 the Repository looks it up; at L5 — the backend. The address stays.
    final record = mockRecords.firstWhere((r) => r.id == id);
    return Scaffold(
      appBar: AppBar(title: Text(record.title)), // the back arrow: automatic
      body: Center(child: Text(record.artist)),
    );
  }
}
```

And the question someone asks every year: “but why send the id and look the object up again — can’t I send the whole object?” You can — `go_router` has the `extra:` parameter — and it is a **trap**. `extra` is not part of the address, so it does not survive exactly the scenarios we adopted address-based routing for: a direct link, a refresh on the web, a state restoration all land on `/records/7` *without* `extra` — and if the detail depends on the received object, it crashes precisely where it was supposed to shine. The healthy idiom has two halves: **the identity travels in the address; the screen looks up its own data.** Today it looks in the mock, at L4 through a `Repository`, at L5 from the backend — the same address, the source changes underneath: one more reason the id in the path is the right investment. The bridge that becomes literal at Chapter 8: this is exactly how a REST endpoint works — `GET /records/7`, not “the object through the session”.

One map nuance, so you do not learn it from an exception: routes are tried top to bottom, so a *fixed* route like `/records/new` is declared **before** the variable `/records/:id` — otherwise “new” is caught by `:id` and `int.parse('new')` crashes. The L3 guide repeats it for you at the scene of the crime.

6.6 The full flow: everything together — exactly what L3 requires

We tie the chapter’s three parts into the two flows you will demonstrate at the L3 defense.

The read flow: list → detail → back. The L2 card, now with `onTap` (6.2), pushes with the id in the address (6.5), the detail looks up its object, and the back arrow — appeared by itself — pops back into the list:

```
Collection ──tap on the card──▶ push('/records/7') ──▶ RecordDetail
     ▲                                                       │
     └──────────────────── ← back (pop) ─────────────────────┘
```

Count the lectures in a single line of code: `onTap: () => context.push('/records/${record.id}')` — the reusable card (L2), the function-value callback (C2), the interpolation (C2), the parameterized address (today).

The write flow: the form that validates, announces, and leaves. On submit: `validate()` first; invalid — `return`, the errors are already red under the fields; valid — short feedback and navigation. And the `go`/`pop` choice is made with the rule from 6.5, on the concrete case:

```
// login_screen.dart — the button that validates and leaves
FilledButton(
  onPressed: () {
    if (!_formKey.currentState!.validate()) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Welcome!')),
    );
    context.go('/'); // go: the login VANISHES from the stack — no way back
  },
  child: const Text('Sign in'),
),
```

After login — `go`: the login leaves the stack. An "add a record" form, opened with `push` from a list, closes elegantly with `context.pop()` instead — you return exactly to the list you came from. In the language of the themes: MediShop — the cart → placing the order with the validated phone → `go` to the order history; EduPlan — the assignment form with the deadline in the future → `pop` back into the list; FitTrack — the new session with the numeric duration validated at 6.4.

The promised mention: the persistent tabs. For the `NavigationBar` some of you chose at L2 — the bottom bar that must *stay* on screen while the content changes — `go_router` has the shell routes (`ShellRoute` and the per-tab-stacks variant you will actually use): the shell stays put, the content switches with `go`. Only this much here: the step-by-step construction, with the map restructuring and the bar wiring, is the material of the L3 guide, Task 4 — we do not duplicate it.

The chapter → lab map, as at the lecture:

The requirement in the L3 guide and themes	The concept that closes it
the theme's navigation: list → detail → form	`GoRoute` + `context.push` + the `:id` parameter (6.5)
the `Navigator.push` scaffolding from L2 gets replaced	the map in `router.dart` + `MaterialApp.router` (6.5)
the static login becomes functional (locally!)	`Form` + `GlobalKey<FormState>` + `TextFormField` (6.4)
the theme's validations: phone, amount, future date	`validator: String?` — the error text or `null` (6.4)
the form that saves and leaves on submit	`validate()` → controllers → `SnackBar` → `go`/`pop` (6.6)

The requirement in the L3 guide and themes	The concept that closes it
feedback: confirmations and decisions	`SnackBar` + a dialog with `await` and `dialogCtx` (6.3)
`NavigationBar` with the main destinations	the shell routes — built step by step in L3

And the hook toward Chapter 7, phrased as a question: today the detail *looked up* its data by id — but if in another screen the user adds the product to the cart, how does the cart screen *know* that something changed? `setState` is local; the controllers are local; `FormState` is local. **Shared** state — the kind that crosses screens — is exactly the material of Chapter 7. Until then, the L3 contract is serene: the forms validate and confirm, but the list does not grow — and that is not a defect, it is the stage.

Chapter summary

Callbacks. Interactivity is a function value (C2) passed to the widget: `onPressed`/`onTap` of type `VoidCallback` (`void Function()`); with a parameter — `ValueChanged<T>`. Pass the *function* (`_submit` or `() => _submit()`), not the call (`_submit()` runs at build; with functions that return functions, the mistake even compiles). `onPressed: null` = disabled — gray, automatic; the `canSubmit ? _submit : null` idiom. Bridges: `addActionListener` (Swing), `Click +=` (C#) — but a constructor parameter, not a subscription.

Gestures. Three steps, in trying order: the dedicated widget (a button, `ListTile.onTap` — ripple + semantics), `InkWell` (tap + the Material ink; requires a `Material` ancestor — `Card`/`Scaffold` provide it), `GestureDetector` (any gesture, any widget — visually mute; long-press, double-tap, drag; for "delete with a swipe" — `Dismissible`). The trap: the ink is painted on the Material *underneath* — an opaque child (the cover image) covers it; the cure for images: `Ink.image`.

Feedback. Just announcing — `SnackBar` through `ScaffoldMessenger.of(context)`: the messenger sits *above* the screens (installed by `MaterialApp`), so the announcement survives navigation. Asking for a decision — `showDialog` → `Future<T?>` + `await` (C3); `null` = dismissed outside → always `ok == true`, never `ok!`. The dialog closes with the *builder's* context (`dialogCtx`) — under nested navigators (L3), `pop` on the screen's context closes something else. After an `await` — the context only under `context.mounted`. Bridges: `JOptionPane`, `MessageBox.Show` — but those blocked the thread; here only your flow waits.

Forms. `TextField` = raw; `TextFormField` = + `validator` + `onSaved` + teamwork under a `Form`. The `TextEditingController` lives in the `State` (not in `build`!) and dies in `dispose` — who creates, releases (C4, `IDisposable`). `Form` + `GlobalKey<FormState>` — the 4.7 promise paid: the global key gives *access* to the state (`currentState`) from anywhere; it is needed because `.of(context)` climbs, and the `Form` is *below* the screen's context. The key idiom: `validator: (value) → String?` — the error text or **`null` = valid**; the error draws itself, red, under the field (the `error` color, C5). `validate()` runs all the validators, the errors bloom simultaneously; invalid → `return`; valid → controllers (or `save()` + `onSaved`) → action. The toolkit: `keyboardType`, `obscureText`, `inputFormatters` (prevents *during* typing) vs `validator` (judges *after*), `textInputAction`,

`decoration`, `maxLength`/`maxLines`. Bridges: Bean Validation, DataAnnotations — except the rule is a function. NOT state management — shared state is C7.

Navigation. The model: the stack — `push` adds, `pop` removes (bridge: `startActivity`). The imperative's limit: the current place *has no name* → no door for a direct link, a URL on the web, restoration. The declarative solution: you describe the *map*, the framework keeps the stack — `go_router`, a package from pub.dev (`flutter pub add`; a ritual repeated at C8/C9/C12; bridge: Maven/NuGet). The map in `router.dart`: `GoRoute(path, builder)`; the `MaterialApp.router(routerConfig:)` wiring **replaces** `home:`; under the hood — still `Navigator`; the map is a global object → changes require `R`, not `r`. **`push` adds** (back returns you); **`go` makes the stack become the address's** (it does not "empty" — it rebuilds after the map; on nested routes, the chain of parents). After login — `go`, mandatorily. Path parameters: `/records/:id` → `state.pathParameters['id']` — always a `String`, the conversion (`int.parse`) is yours; the `!` justified by the map. `extra` carries objects but dies on a direct link/refresh/restoration — the healthy idiom: **the id travels, the screen looks it up** (today mock, L4 Repository, L5 backend; like `GET /records/7`). Fixed routes before variable ones.

Self-assessment exercises

The rules you know: solve them without the material in front of you, ideally with DartPad open — but do not run anything before answering; the hints at the end show *where* to look, not the solution. The exercises verify the mechanism — applying it to your theme remains the L3 guide's business and is not duplicated here.

1. *(code prediction)* For a screen with the method `void _save() { … }`, say about each variant whether it compiles and what it actually does: (a) `onPressed: _save`; (b) `onPressed: _save()`; (c) `onPressed: () => _save()`; (d) `onPressed: null`. Then the subtle case: the method `VoidCallback _makeSaver() { debugPrint('preparing'); return _save; }` and `onPressed: _makeSaver()` — does it compile? when does the `debugPrint` run, and how many times?

2. *(a trap to identify)* A colleague's card: an `InkWell` with a working `onTap`, and inside it an `Image.asset(..., fit: BoxFit.cover)` over the whole surface. The tap works, but no ripple is ever seen — and no error anywhere. Explain the mechanism (where the ink is painted and who covers it), give the dedicated repair for images, and say where the `InkWell` had its `Material` ancestor from.

3. *(code prediction)* A form shows `ScaffoldMessenger.of(context).showSnackBar(...)` and, on the very next line, `context.pop()`. Does the announcement survive the closing of the screen? Why — who keeps it alive, and where does that object sit in the tree? What would happen, by contrast, if the announcement belonged to the screen's `Scaffold`?

4. *(validators to write)* Write one `validator` each (3–6 lines apiece) for: (a) a mandatory phone, exactly 9 digits — knowing that the field already has `FilteringTextInputFormatter.digitsOnly`: what remains to check, and what is no longer worth checking?; (b) a positive amount with decimals (`double.tryParse`); (c) an optional e-mail field: empty = accepted, but if filled in, it must contain `@`. For each: what does the returned `null` mean?

5. *(find the errors)* In the code of a confirmation dialog, a colleague wrote `onPressed: () => Navigator.pop(context, true)` on the "Yes" button (with the screen's context, not the builder's) and then `if (ok!) { … }` on the result. Both "work" in his quick test. Explain the two time bombs: under what conditions each one blows up, and how they are written correctly.

6. *(stack prediction)* The application starts on `'/'` (Collection). The user: `push('/records/7')`, then `push('/records/9')`, then presses back, then `go('/login')`, then `push('/records/3')`. Draw the stack after *each* step. At the end the user presses back — where do they land, and why not in Collection?

7. *(stack prediction — login)* After the successful authentication, two variants: `context.push('/records')` or `context.go('/records')`. Draw the stack in both cases and say what the back button does in each. Which variant does the L3 grading scheme require, and what is the *correct* phrasing of `go`'s effect (careful: not "it empties the stack")?

8. *(the extra trap)* A colleague sends the whole object to the detail: `context.push('/records/7', extra: record)`, and the detail reads `state.extra as VinylRecord`. In the demo it works perfectly. List at least three scenarios in which the application crashes or shows an empty screen, explain what all three have in common, and phrase the healthy idiom — including why it stays valid when the data source changes at L4 and L5.

9. *(a map trap + types)* In the map, `/records/:id` is declared before `/records/new`, and the builder does `int.parse(state.pathParameters['id']!)`. What happens on `context.push('/records/new')` — and why? How is it repaired? Then: why is `pathParameters['id']` a `String` even though all your ids are numbers — and what does the `!` after it guarantee?

10. *(hands-on, synthesis — no go_router, works in DartPad)* Build a mini-form: the fields "Title" (required) and "Duration, minutes" (a positive whole number), plus a "Save" button. Requirements and checks: (a) the controllers — where do you declare them, where do you release them, and what would break if you created them in `build`?; (b) press "Save" with both fields wrong — how many errors appear *simultaneously*, and who draws them?; (c) make the button disabled while an "I accept the terms" `Checkbox` is unchecked (the `condition ? function : null` idiom); (d) on a valid submit, show a `SnackBar` with the title — why through `ScaffoldMessenger` and not through a "message" widget of your own?

Hints

- **1:** 6.2, the parentheses trap: reference vs call; `_save()` is `void` — it does not compile; `_makeSaver()` returns a function — it compiles, and the body runs on *every* build; (d) — the disabled button.
- **2:** 6.2, `Ink.image`: the ink is painted on the Material underneath; the opaque image covers it; the Material comes from `Card`/`Scaffold`.
- **3:** 6.3 — the messenger is installed by `MaterialApp`, *above* the screens; that is why the announcement lives on after the `pop`.
- **4:** 6.4 — the `null` = valid idiom; in (a), the formatter has already stopped the letters: the length and the requiredness remain; in (c), empty → `return null` *early*.

- **5:** 6.3 — `dialogCtx` vs the screen's context (nested navigators at L3) and `ok == true` vs `ok!` (a dialog dismissed outside yields `null`).
- **6:** 6.5, the `push`/`go` diagrams; do not forget: `go('/login')` makes the stack become the address's — Collection is no longer underneath.
- **7:** 6.5, the textbook login case; the correct phrasing: the stack *becomes the one the address describes in the map*.
- **8:** 6.5, the `extra` trap: a direct link, a refresh on the web, state restoration — all land on the address *without* `extra`; the id travels, the screen looks it up.
- **9:** the end of 6.5: routes are tried top to bottom — the fixed one before the variable one; an address is *text*, so the parameter is a `String`; the `!` — the map's guarantee on that route.
- **10:** all of 6.4 + 6.2 (the `null` idiom) + 6.3 (`ScaffoldMessenger`); for (a) — 4.4: `build` runs often, the controller would lose its text.